API Security Risk Analysis Report

Cyber security Internship -TASK 3

Internship Program: Future Interns

API Tested: JSONPlaceholder (Public REST API)

Submitted By: shashank

Date: 01 March 2026

Introduction

This report presents a security risk analysis of a publicly available demo API, JSONPlaceholder. The objective of this assessment is to Identify potential security weakness in API endpoints and evaluate their impact if implemented in areal-world production environment.

APIs play a critical role in modern web and mobile applications by enabling communication between client and server systems. Securing APIs is essential to prevent data breaches, unauthorized access, and service abuse.

API Selected for Testing

API Name: JSONPlaceholder

Base URL: https://jsonplaceholder.typicode.com

JSONPlaceholder is a public REST API used for Testing and prototyping.

Tools Used:

- Postman (API Testing Tool)
- Web Browser
- Screenshot Documentation
- GitHub (for report hosting)

Methodology

The following approach was used during the testing:

1. Sent GET requests to available endpoints.
2. Observed response structure and status codes.
3. Checked for authentication requirements.
4. Modified object IDs to test authorization controls.
5. Sent repeated requests to test rate limiting.
6. Analyzed exposed data fields.

All testing was conducted in a non-destructive and read-only manner.

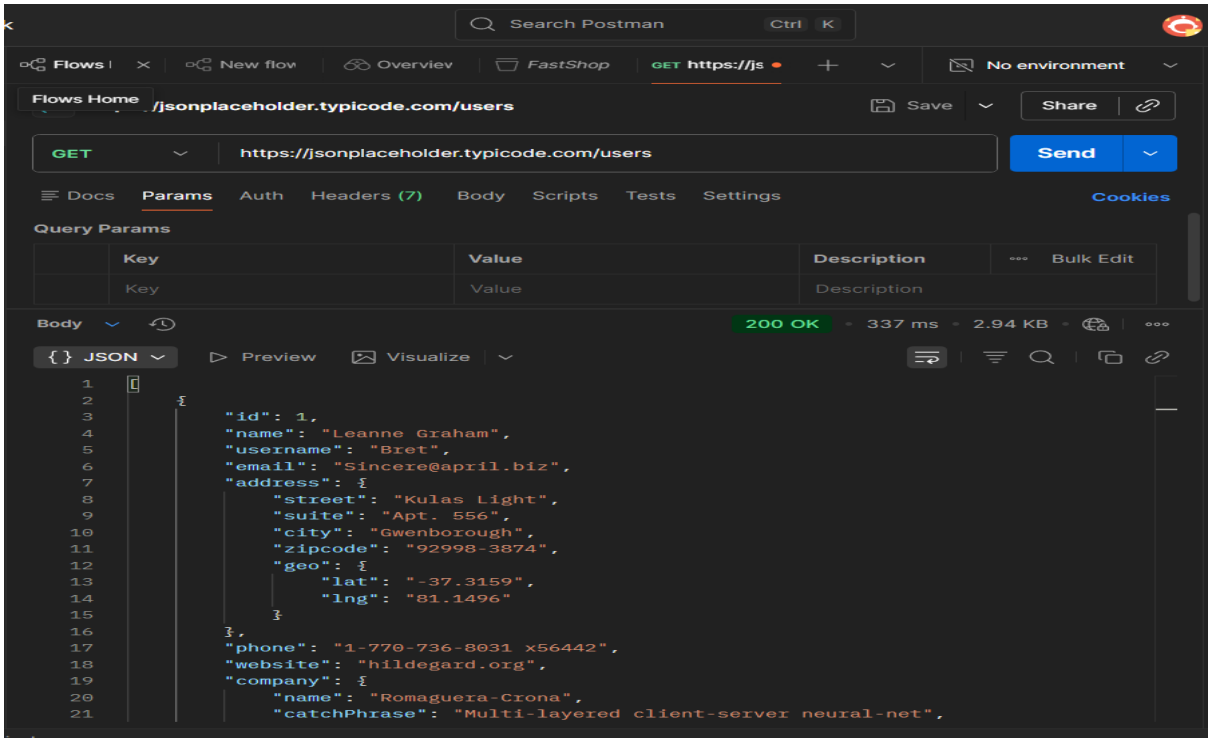
Identified Security Risks

Risk 1 – Missing Authentication

Endpoint Tested: `GET /users`

Description: The API allows access to user data without requiring authentication or Authorization tokens.

Proof of concept (PoC):



Impact: In real-world environment, unauthenticated access could allow attacker to retrieve sensitive user information.

Severity: Medium

Remediation: Implement secure authentication mechanisms such as JWT or OAuth 2.0

Risk 2 – Broken Object Level Authorization (IDOR)

Endpoints Tested:

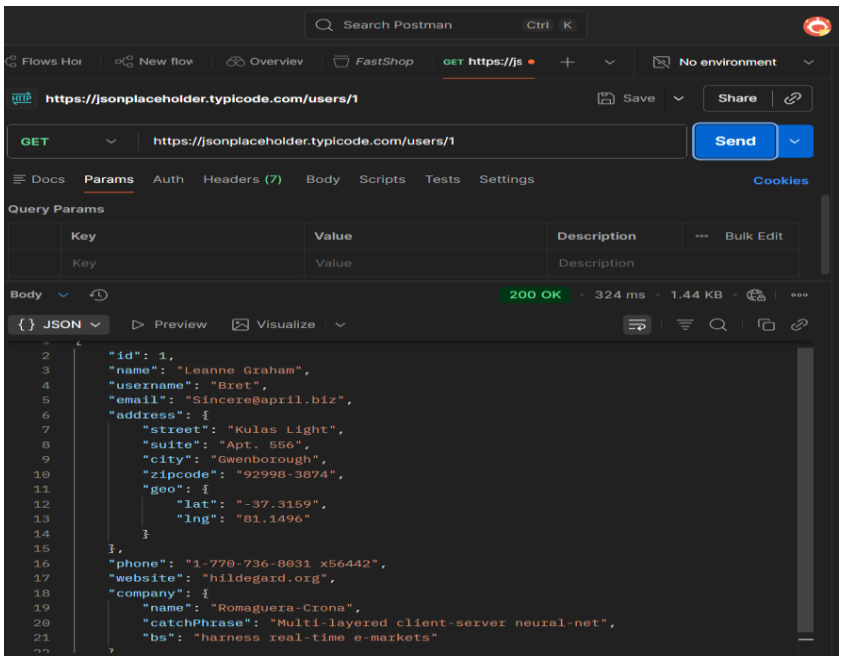
Get /users/1

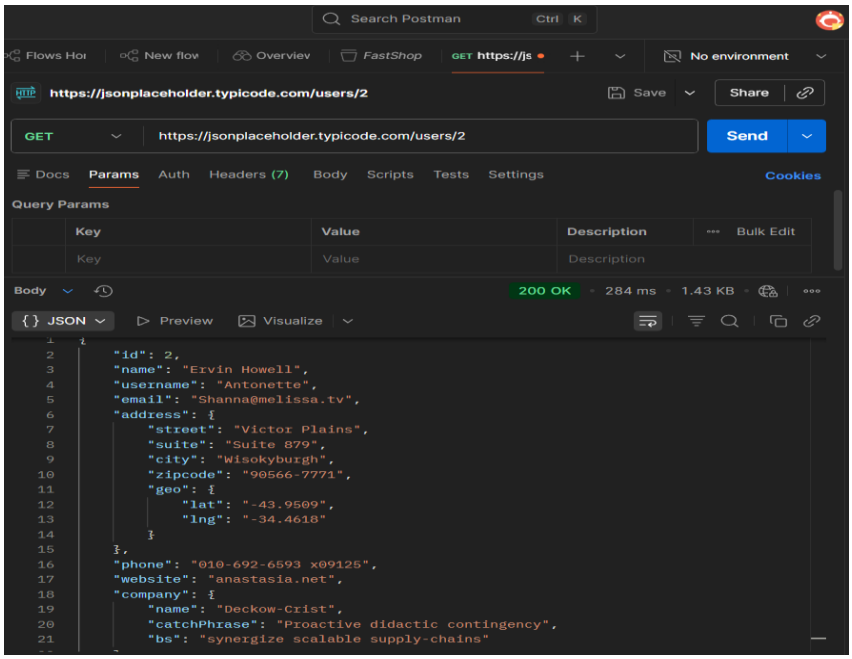
Get /users/2

Description:

By modifying the user ID in the URL, different user records can be accessed without any access control validation.

Proof of concept (PoC):





Impact: In a production system, this could allow users to access or manipulate other users' private information.

Severity: High

Remediation: Implement proper authorization validation and role-based access control (RBAC).

Risk 3 – Excessive Data Exposure

Description: The API response exposes detailed user information including full address, geographic coordinates, phone numbers, and company details.

Proof of concept (PoC):


```
{
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
    "email": "Shanna@melissa.tv",
    "address": {
        "street": "Victor Plains",
        "suite": "Suite 879",
        "city": "Wisokyburgh",
        "zipcode": "90566-7771",
        "geo": {
            "lat": "-43.9509",
            "lng": "-34.4618"
        }
    },
    "phone": "010-692-6593 x09125",
    "website": "anastasia.net",
    "company": {
        "name": "Deckow-Crist",
        "catchPhrase": "Proactive didactic contingency",
        "bs": "synergize scalable supply-chains"
```


Impact: Excessive exposure of data may assist attackers in profiling users or conducting targeted attacks.

Severity: Medium

Remediation:

Return only necessary fields required by the application.

Risk 4 – Missing Rate Limiting

Description: Multiple rapid requests were sent to the same endpoint. The API continued responding with "HTTP 200 OK" without returning any rate-limiting errors such as 429 (Too Many Requests).

Impact: In real-world applications, lack of rate limiting could allow brute-force attacks, data scraping, or denial-of-service (DoS) attempts.

Severity: Medium

Remediation: Implement request throttling and rate limiting per IP address or user session.

Conclusion: The security analysis identified several common API security weaknesses including "missing authentication, broken authorization, excessive data exposure, and absence of rate limiting".

Although JSONPlaceholder is a demo API intended for testing purposes, these findings demonstrate how similar vulnerabilities in production environments could lead to serious security incidents.

Proper authentication, authorization controls, data minimization, and traffic management mechanisms are essential to ensure secure API implementation.

Security Recommendation:

Broken Object Level Authorization (IDOR):

- Implement Token-Based Authentication (JWT or OAuth 2.0).
- Enforce Centralized Authentication Middleware.

Excessive Data Exposure:

- Apply Data Minimization Principle.
- Implement Response Filtering/Serialization Controls.

Missing Rate Limiting:

- Implement Rate limitation via API Gateway.
- Return HTTP 429 (Too many requests) with monitoring Alerts.